DHBW Stuttgart

Karsten Keßler

Datenbanksysteme
12. Teil – Sichten und Rechte

Kurs Informatik 24ITA

Inhalte der heutigen Vorlesung

- Definition von Sichten
- Herausforderungen und Möglichkeiten
- Sichten in SQL
- Weitere Konzepte in SQL
  - → Rechte, Rollen, Sicherheit

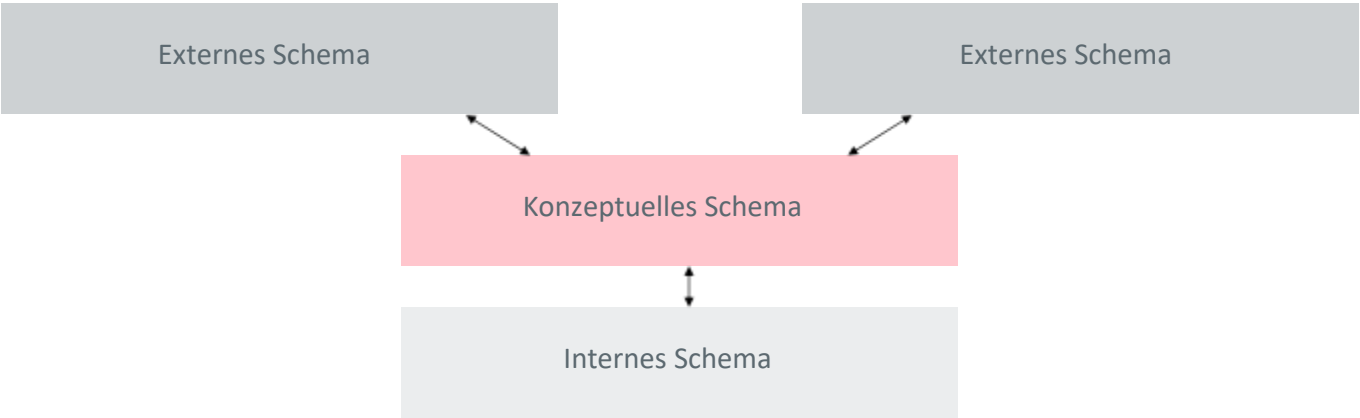
Wiederholung – Schema-Architektur

Konzeptuelle Ebene **Konzeptuelles Schema**	Ergebnis aus Datenmodellierung, Datenbankentwurf und Datendefintion
Externe Ebene **Externes Schema**	Ergebnis der Sichtdefinition, Sichten auf globales konzeptuelles Schema
Interne Ebene **Internes Schema**	Dateiorganisation und Zugriffspfade

Wiederholung – Ebenen in SQL und Schemaarchitektur

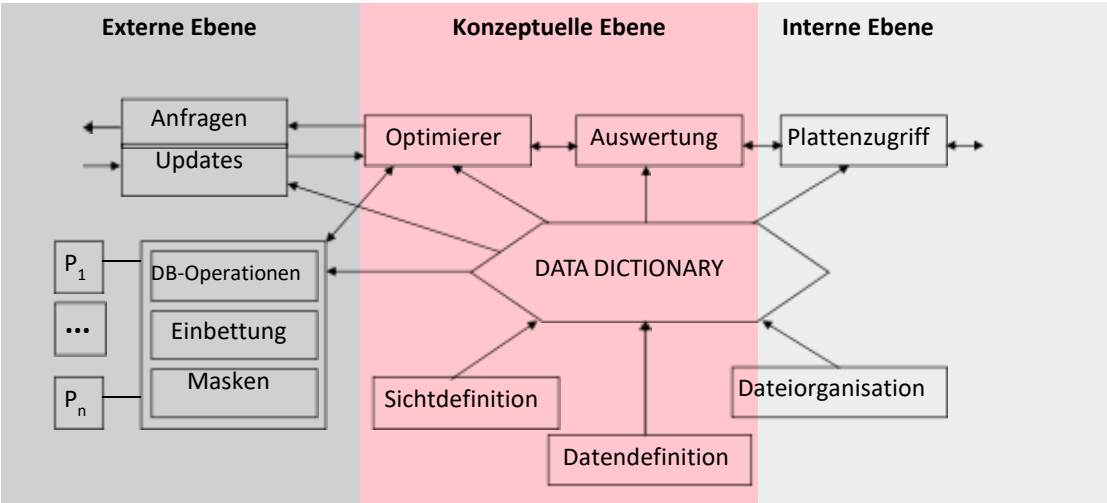
Konzeptuelle Ebene	CREATE TABLE <TabellenName> (<attrib-name> <Wertebereich>, ...) DROP TABLE <TabellenName> ALTER TABLE <TabellenName> [...]
Externe Ebene	CREATE VIEW <Sichtname> [<Schemadeklaration>] AS <SQL-Anfrage> DROP VIEW <Sichtname>
Interne Ebene	CREATE INDEX <IndexName> ON <TabellenName> (<indexcol> ...) DROP INDEX <IndexName> ALTER INDEX <IndexName> [...]

Drei-Ebenen-Schemaarchitektur für Datenbanken



Mittel zum Erreichen der **logischen Datenunabhängigkeit**

Externe Ebene – Sichtdefinition – Sichten auf globales konzeptuelles Schema



Definition und Motivation

- Eine Sicht ist eine **virtuelle** oder **berechnete** Datenbank, keine **tatsächliche**!
  - → Wird aus der tatsächlichen durch eine feste Berechnungsvorschrift abgeleitet
- Zweck
  - → Daten ausblenden
  - → Daten in anderer Form darstellen
- Sicht = gesamte virtuelle Datenbank oder Element einer abgeleiteten Datenbank
  - → Manche Nomenklaturen: nur eine virtuelle Relation
- Hier: relationale Sichten, Bestandteil der Sprache SQL

Sichten: Definition

Eine Sicht besteht aus:

A) Einem **Relationenschema**
B) Einer **Berechnungsvorschrift**

} + ein **Name** für die Sicht

Definition einer Sicht in SQL

```
CREATE VIEW SichtName [Schema]
AS  { ... }
[WITH CHECK OPTION]
```

← Prüfen vor Updates

Zum Löschen → DROP VIEW SichtName

Vorteile von Sichten

+ Vereinfachung von Anfragen
+ Datenbankschemata können groß, komplex und für den Einzelanwender undurchsichtig werden.
  → Benutzerbezogene Strukturierung
+ Stabilität der Anwenderschnittstelle bei Datenbankstrukturänderungen
  → Logische Datenunabhängigkeit
+ Zugriffskontrolle

Sichten: Beispiel

Prüfungsdaten **Prüf**(Studienfach, Fach, Student, Prüfer, Datum, Note)

Sichten

Die Informatik Fakultät sieht nur die Daten der Informatikstudenten

Das Prüfungsamt sieht alle Daten

Jeder Student darf seine eigenen Daten sehen (aber nicht ändern)

Die Kommission für die Vergabe von Promotionsstipendien darf von den Studenten die Durchschnittsnote sehen

Der Dekan darf statistische Daten über die Absolventen des letzten Jahrgangs lesen

Die Sekretariate dürfen die Prüfungsdaten der zugehörigen Professoren einsehen

Probleme mit Sichten

Umsetzung der Sicht wird vom DBMS automatisch vorgenommen

→ Anfragen und Änderungen werden automatisch von der Sicht auf die „echte“ Datenbank umgesetzt

→ Änderungen auf Sichten können nicht immer auf sinnvolle Änderungen der „echten“ Datenbank übertragen werden.

Arten von Sichten

A. Projektionssichten

B. Selektionssichten

C. Verbundsichten

D. Aggregierungssichten

DHBW Stuttgart

Beispielschema:
MGA(Mitarbeiter, Gehalt, Abteilung)
AL(Abteilung, Leiter)

A. Projektionssichten – Ausblenden von Spalten

CREATE VIEW MA as SELECT Mitarbeiter, Abteilung FROM MGA

Löschen eines Tupels? Kein Problem - Löschen in der Datenbank durchführen?

Mengeneigenschaftsproblem: Ein Eintrag für mehrere Einträge in der DB!

Einfügen eines Tupels? Einfügen von Nullwerten für die ausgeblendeten Attribute

INSERT INTO MA VALUES (‚Zuse‘, ‚Info‘)

transformieren → INSERT INTO MAG VALUES (‚Zuse‘,NULL,‚Info‘)

Konsistenzverletzung?

DHBW Stuttgart

Beispielschema:
MGA(Mitarbeiter, Gehalt, Abteilung)
AL(Abteilung, Leiter)

B. Selektionssichten – Ausfiltern von Zeilen

MG:= $SEL_{Gehalt>20}$ (PROJ $_{Mitarbeiter,Gehalt}$(MGA))

CREATE VIEW MG AS SELECT Mitarbeiter, Gehalt FROM MGA WHERE Gehalt > 20

WITH CHECK OPTION

Änderungsaktion:

UPDATE MG
SET Gehalt = 15
WHERE Mitarbeiter = ‚Zuse‘

transformiert →

UPDATE MGA
SET Gehalt = 15
WHERE Mitarbeiter = ‚Zuse‘ AND Gehalt > 20

*Saake/Heuer zitieren

DHBW Stuttgart

Beispielschema:
MGA(Mitarbeiter, Gehalt, Abteilung)
AL(Abteilung, Leiter)

C. Verbundsichten – Zusammenstellen einer virtuellen Relation

MGAL:= MGA JOIN AL

```
CREATE VIEW MGAL AS
SELECT Mitarbeiter, Gehalt, MGA.Abteilung, Leiter
FROM MAG, AL
WHERE MGA.Abteilung = AL.Abteilung
```

Einfügen: INSERT INTO MGAL VALUES ('Turing', 30, 'Info', 'Zuse')

transformiert →

INSERT INTO MAG VALUES ('Turing', 30, 'Info')

Falls Zuse nicht schon als Leiter von Info eingetragen ist → INSERT INTO AL VALUES (,Info', ,Zuse')

Falls Zuse bereits die Abteilung Mathematik leitet → UPDATE AL SET Abteilung = ,Info' WHERE Leiter = ,Zuse'

Alle MA der Abteilung Mathematik haben nun keinen Leiter mehr ...

DHBW Stuttgart

Beispielschema:
MGA(Mitarbeiter, Gehalt, Abteilung)
AL(Abteilung, Leiter)

C. Verbundsichten (Joins) – Zusammenstellen einer virtuellen Relation (2)

DELETE FROM MGAL
WHERE Mitarbeiter = 'Turing' AND Gehalt = 30 AND Abteilung = 'Info' AND Abteilungsleiter = 'Zuse'

transformiert →

DELETE FROM MAG
WHERE Mitarbeiter = 'Turing' AND Gehalt = 30 AND Abteilung = 'Info'

Oder?

DELETE FROM AL
WHERE Abteilung = 'Info' AND Abteilungsleiter = 'Zuse'

Oder? Beides ...?

→ Explizite Regeln für die Löschung in Verbundsichten formulieren

SQL 92 untersagt Änderungen auf Verbundsichten

Beispielschema:
MGA(Mitarbeiter, Gehalt, Abteilung)
AL(Abteilung, Leiter)

D. Aggregierungssichten: durch Gruppierung oder Aggregat

```
CREATE VIEW AS (Abteilung, SummeGehalt)
AS
SELECT Abteilung, SUM(Gehalt)
FROM MGA
GROUP BY Abteilung
```

Die Gehaltsausgaben der Abteilung ‚Info' sollen nun um 1000 EURO erhöht werden ...

```
UPDATE AS
SET SummeGehalt = SummeGehalt + 1000
WHERE Abteilung = ‚Info'
```

transformiert → ? Keine sinnvolle Übersetzung möglich!

Allgemein: Herausforderungen bei der Verwaltung von Sichten

Integritätsverletzungen	Beispiel: Einfügen von Nullwerten, die nicht zulässig sind
Zugriffsschutz	Seiteneffekte auf nicht-sichtbaren Bereich
Auswahlproblem	Mehrere Transformationsmöglichkeiten
Sinnhaftigkeit	In manchen Fällen ist keine sinnvolle Transformation in Änderungen im zugrundeliegenden Modell möglich

1:1 Beziehung zwischen Sichttupeln und Tupeln der Datenbank

Dies ist bei „Herausprojizieren" eines Schlüsselattributs nicht gegeben.

Behandlung von Sichten in SQL

Konventionen:

Zurückweisen integritätsverletzender Änderungen in Sichten (z.B. NOT NULL)

Administrationskontrolle: „Check Option“ für sicherheitsverletzende Änderungen

Ist eine nicht-eindeutige Transformation möglich, so ist die Sicht nicht änderbar (welche Basistabelle betroffen?)

Elementare Sichtänderungen = elementare Basisrelationsänderungen (muss sich 1:1 auf Tabelle zurückführen lassen, keine Berechnungen, keine Joins, keine Aggregatfunktionen)

Auswertung: „Mischen“ von Anfrage und Sichtdefinition

SQL 92 : Expansion des Sichtnamens im FROM Teil der Anfrage

Ursprüngliche SQL-Norm:

1. SELECT: Wenn nötig umbenennen der Sichtattribute (ggf. durch Berechnungsterm ersetzen)
2. FROM: Namen der Originalrelationen aufführen
3. WHERE: Konjunktive Verknüpfung der WHERE-Klauseln von Anfrage und Sichtdefinition

Beispielschema:
MGA(Mitarbeiter, Gehalt, Abteilung)
AL(Abteilung, Leiter)

Beispiel View Extension

CREATE VIEW **DS** (Abteilung, GehaltSumme) AS
SELECT Abteilung, sum(Gehalt)
FROM MGA
GROUP BY Abteilung

SELECT Abteilung FROM **DS** WHERE GehaltsSumme > 500

Übersetzt in: SELECT Abteilung FROM MAG WHERE SUM(Gehalt) > 500 GROUP BY Abteilung

SELECT avg(GehaltsSumme) FROM DS

Übersetzt in: SELECT avg(sum(Gehalt)) FROM MAG GROUP BY Abteilung

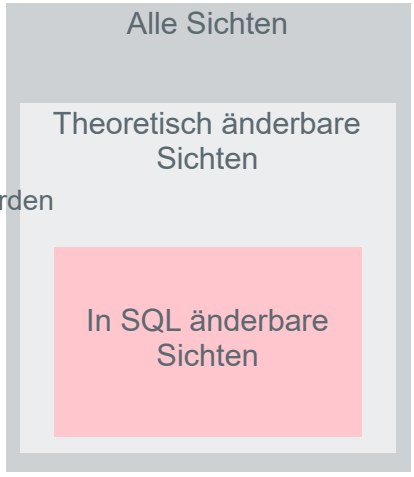
→ Views sind **nur scheinbar** Tabellen – SQL löst sie auf und baut die Anfrage neu zusammen.

SQL 92 und Sichten

Änderungen sind nur erlaubt, wenn:

- die Anfrage eine reine Selektionsanfrage ist
- die Anfrage zur Sichterzeugung kein DISTINCT enthält
- keine Arithmetik und Aggregatfunktionen im SELECT-Teil verwendet werden
- nur eine Basisrelation, d.h. eine Referenz im FROM-Teil verwendet wird
- keine Unteranfragen mit Selbstbezug vorhanden sind
- keine Gruppierungen (GROUP BY / HAVING) vorhanden sind

Solche Sichten werden in SQL 92 als **„updatable“** geführt



Rechtevergabe in SQL-92

Freigeben von Ausschnitten der Datenbank für bestimmte Nutzer/Nutzergruppen.
Freigeben von bestimmten *Operationen* auf die Datenbanktabellen.

GRANT <die ausführbaren Operationen>
ON <Sicht/Tabelle>
TO <Benutzer/Benutzerliste>
[WITH GRANT OPTION]

REVOKE <Rechte> ON <Tabelle>
FROM <Nutzer> [RESTRICT / CASCADE]

Insert / Select / Update / Delete

Recht auf Weitergabe von Rechten

Beispiel: „public“ - View

```
CREATE VIEW  MeineVorlesung AS
SELECT * FROM Vorlesung WHERE Prof = user;

GRANT select, insert ON MeineVorlesung TO public;
```

Jeder Dozent kann seine Vorlesungen einsehen

Vielen Dank für Ihre Aufmerksamkeit!